

Instructor Handout 1: Where Does Your Data Live?

Pointers, Dynamic Memory, and Simple Contracts

Dr. Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 12, 2026

Instructor Copy — Not for Student Distribution

1.1 Before You Teach This Lecture

- **Wirth, *Algorithms and Data Structures*, §4.2, p.111** — the pointer / dynamic-allocation anchor. §4.1 (p.109) gives the recursive-variability motivation. *Page-cited*; index built 2026-08-11. Read the §4.2 opening paragraph if you only read one thing.
- **Sedgewick & Wayne, *Algorithms*, §1.2, p.64** — the ADT/data-type definition; §1.3, p.120 — the bag as a collection type; §1.4, p.172 — the “how long will my program take” motivation for cost analysis. *Page-cited*. All are in Java; convert syntax to C per the notation registry, never present it as C.
- **Karumanchi, *Data Structures And Algorithms Made Easy*, Ch. 1, p.18–19 (PDF)** — the ADT definition (“a declaration of data” plus “a declaration of operations”) and its list of commonly used ADTs, used here to preview the Weeks 3–12 roadmap. *Page-cited*; index built 2026-08-11. **Not** a source for this lecture’s pointer/malloc content — indexing found zero coverage of structured programming or `malloc/calloc/realloc/free` in this book.
- **Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 1 and Aho–Hopcroft–Ullman, Ch. 1** — *not page-indexed*. Cite at chapter level for structured programming / the ADT framing.
- **Scope notice:** this lecture deliberately defers `calloc` and `realloc`. Allocation vocabulary is only `malloc`, `sizeof`, `free`, `NULL`. Do not introduce the other two here even if students ask — it dilutes the one-block story.
- **Plan the board examples in advance** (see each section below, marked **Board:**). The two running threads — the stack trace and the heap trace — are the ones to rehearse; the numbers on the board must be right the first time.

1.2 Starting Point: Three Questions

Open by stating the question the whole lecture answers: *where does a running C program keep its data?* Write it at the top of the board and leave it there — every section of this lesson is one corner of that answer.

Pose the three questions on the board before revealing anything: (1) where does a running C program keep its data, (2) how can we use memory we request while the program runs, (3) how can we describe a data structure before implementing it. Let students guess before you structure them.

Board: three quick prediction boxes. Draw three labelled boxes (“where,” “how,” “describe”) and invite one-word answers from the class; leave the boxes unfilled until the end and return to them in the summary. This is the lesson’s live spine and costs nothing.

Teaching note: The trap to avoid on the first frame is jumping straight into vocabulary. The “A Variable Has a Place” idea — `int marks = 80` has a name, a value, *and* an address — is the seed of everything; spend the time making the address concrete before naming the stack. Use the house-number line: “an address is like a house number; it tells us where to look.”

Board: a variable and its place. Draw one box for `marks`, put 80 inside it, and write an invented address (say `0x7FFC`) above the box. Ask: “if we only knew the value, could we tell the box from its neighbour? What else do we need?” Answer: the address — which is exactly what a pointer stores.

1.3 Motivation and the Overall Syllabus (new Act 0)

This rebuilt deck opens with a short Act 0 that answers *why* before *what*. Its job is motivational, not technical — nothing here is examined directly, but it sets the frame the rest of the term leans on.

Motivation: correct is not always fast enough. Use the student-record story from the Notes: a loop that scans records is correct at 5 records and unusable at 10 million. The point to land: *the answer is the same; the arrangement of the data is the difference*. This is the single sentence that justifies the whole course. **Board:** write “5 records — instant” and “10,000,000 records, a query every millisecond — unusable” on two lines, then draw an arrow between them labelled “same correct loop.” Ask: “what changed?” — not the loop, the arrangement.

Motivation: “it felt fast” is not an argument. Sedgewick’s §1.4 opener (p.172) poses the two questions “how long will my program take?” and “why does my program run out of memory?” as too vague to answer precisely without a model. Land the contrast: Week 1 gives the raw material (memory layout, pointers), Week 2 gives the model (counting operations, $O/\Omega/\Theta$). This frame makes Week 2 feel like the payoff of Week 1 rather than a new topic.

The course in one roadmap. Walk the six clusters: Weeks 1–2 foundations; 3–5 linear structures (arrays, stacks, queues, expression processing); 6–7 linked structures; 8–9 searching and sorting; 10 files and hashing; 11–12 trees, BSTs, AVL, graphs, MSTs, shortest paths. Emphasise the recurring shape: *state the contract, then implement it, then analyse what it costs* — the same three beats every week repeats.

Teaching note: Students commonly ask “why are we learning memory stuff in a data-structures course?” The roadmap slide is the honest answer: every later structure is a claim about *where data lives and how it is reached*, so the memory model is not a prerequisite — it is the subject. Keep this slide to under two minutes and come back to it at the Week 1 summary.

The running thread: the expression processor. Introduce the small program that reads `(a+b)*c` and must check its brackets, convert it, evaluate it, and look up its identifiers. Map each future week onto a deficiency it fixes: a stack checks the brackets (Weeks 3–4); a symbol table, first a list (Week 6) then a hash table (Week 10), stores the identifiers; an expression tree evaluates it

(Week 11). This is the same “one program learning new tricks” thread the Week 0 deck previewed; state it once here and it pays off all term.

What you already know. The bridge slide: variables, arrays, functions, loops, **struct** — all from prior courses. The single new thing this course adds is the question “what arrangement of data fits this job?” and the discipline of answering it with a contract, an implementation, and a cost.

1.4 Memory: The Stack and the Heap

State the two-part geography and keep the orientation fixed all term: the **stack** (short-lived local variables) and the **heap** (memory requested at run time). Figure 1.1 is the simple course picture.

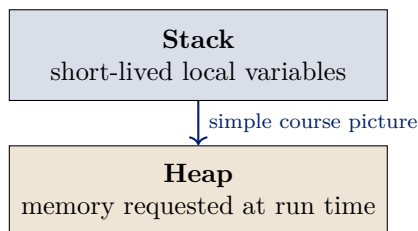


Figure 1.1: The two places a running C program keeps data. Orientation (stack above, heap below) is fixed for the whole course.

Board: draw the two boxes first (stack top, heap bottom), label the growth directions *only if* a student asks — the detailed growth is the next move.

1.4.1 The stack grows (Questions worth the whole story)

The deck’s step-frames (Figures 1.2–1.3) make automatic storage a three-step story:

1. **main** starts $\Rightarrow$ one frame holding `int a = 5;`.
2. `add(x=5, y=3)` is called $\Rightarrow$ a second frame is pushed *below*, holding the parameters and `int s = x + y; = 8.`
3. `add` returns $\Rightarrow$ its frame is popped, and 8 was copied back into **main**’s frame.

Board: live stack trace (interactive). Draw the empty board frame; ask a student to “call” `add` by walking to the board and pushing a new frame below **main**’s. Then a second student “returns” and erases it, copying 8 up. Two volunteers, three steps, and the whole class watches the stack grown and shrink. Ask mid-way: “*where does the next frame go — higher or lower address?*” (Lower — the stack grows down.)

Board: a second, distinct trace (prediction trap). Pose a different call: **main** holds `int n = 3;` and calls `doubleIt(n)` whose local is `int d = 2*n;`. Ask the class to draw the two-frame picture *before* you do, including which address region is higher. Reveal after a show of hands.

Teaching note: The recurring student error here is treating the stack as a “magic box that grows.” The teaching move is the low-address line: the *directions* (heap up, stack down) get drawn once and reused for 12 weeks; if you vary them you undo a later diagram. Also be explicit that *automatic* means “the program manages it with no instructions from you” — students conflate “automatic” with “I don’t have to think about it,” which is exactly wrong when pointers enter.

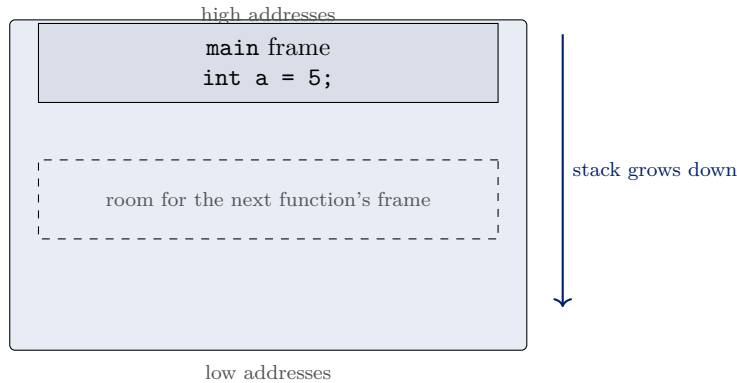


Figure 1.2: Stack, Step 1: `main` starts and its frame is created holding `int a = 5;`.

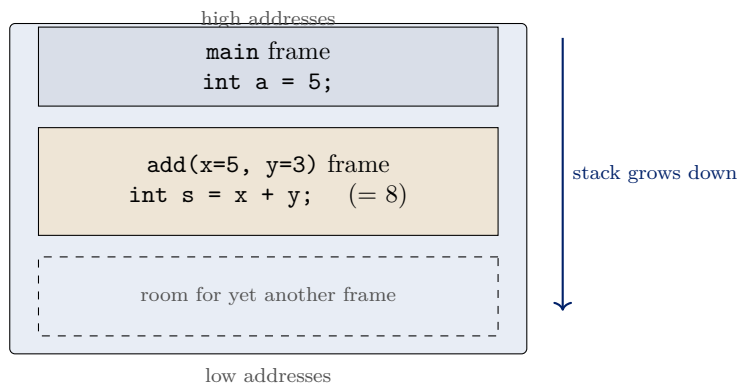


Figure 1.3: Stack, Step 2: the call to `add` pushes a second frame at a lower address. Returning (Step 3) pops it.

1.4.2 Why the stack is not enough

State the requirement the stack cannot meet: the user enters the record count only after the program runs, and we may need 3 records today, 300 tomorrow. The stack decides frame sizes at compile time and discards locals on return — it fails on size *and* lifetime.

Board: the guessing trap. Write `int a[1000000];` on the board and ask: “when is this wasteful, and when does it still fail?” (Wasteful when the count is small; fails when it exceeds a million — so guessing rescues nothing.) This reuses Wirth’s p.111 argument without the formalism: variable-sized data needs storage allocated *when the data comes into existence* [3].

Board: the prediction question (live). Put the two declarations on the board:

```
int score = 80;   vs.   int *p = malloc(sizeof(int));
```

Ask: “*which variable can outlive the function that created it?*” Let votes land. Answer: the *block*, not the variable that names it — the pointer is only the handle. This sets up the entire next two sections. Do not reveal `malloc` detail here; you told them the call exists, and that is the hook.

1.5 Pointers: The Handle on Memory

Definition (Pointer). *A variable that stores an address. Its type records what kind of object resides at that address [3].*

The contrast to draw: an ordinary variable stores a value such as 80; a pointer stores a *place*. `int *p;` means “`p` can point to an `int`.” Every pointer is a handle, not the data.

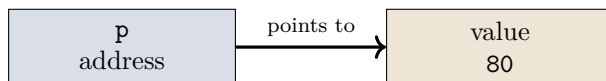


Figure 1.4: A pointer picture: `p` holds an address; the value 80 lives at that address.

Board: box-and-arrow. Draw the two boxes, put 80 in the right one, then draw the arrow and label it “points to.” Ask: “*what does `p` contain — the 80 or the address?*” (The address.) The arrow is the whole abstraction.

1.5.1 Pointers reach structures and stop

Introduce the two-field node:

```
struct node {  
    int data;  
    struct node *next;  
};
```

`data` stores the value; `next` stores the address of *another* node; if there is no next node, `next = NULL`. Establish the arrow field-access now: `p->data` reads the `data` field of the node `p` points at. (The deck defines `->` at first use.)

Definition (NULL). *The address value meaning “points at nothing.” Reading through it is undefined behaviour — in practice, a crash.*

Board: node-picture drill (interactive). Draw three boxes — `head` on the board’s left, then two node boxes, `data/next` each. Ask the class to draw the arrows so that `head` points at node 1 and node 1 points at node 2, with node 2’s `next` written `NUL`L. Then erase the middle arrow and ask: “*what happens to node 2 now?*” Answer: reachable no longer — a preview of how a **dangling** situation begins.

Board: second distinct instance. Instead of node 2, ask the class to extend to a third node and mark where the chain terminates — then give the one-line meaning of `struct node *head = NULL`; (“the list is empty”). Two arrows, labels, one `NUL`L: a cheap repeat that fixes the convention.

Teaching note: Students will write `((*p).data)` from an earlier course and will resist `p->data` as “sugar.” Hold the line: the course standard is `->` everywhere, and the convention registry enforces it. Also expect the “is a pointer a number?” question — yes, an address is a number, but its *type* records what lives there; that is why `int *` and `struct node *` are different types.

1.6 One Heap Block: Allocate, Use, Return

The whole memory discipline of the course is three moves, done to one block at a time. Restate the lesson’s spine before this section: **request memory, use it, return it**. Section 1.7 below is the worked core; Section 1.8 is the growth picture; Section 1.9 is the return and the two mistakes.

1.7 Requesting One Integer

State the pattern before any explanation:

```
int *p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 80;
```

Three facts: `malloc` requests bytes from the heap; `sizeof(int)` asks C for the right number of bytes; `p` stores the address of the requested space. Emphasise that you never count bytes by hand.

Board: the pattern with a distinct instance. Walk the same three lines but for `double` instead of `int`: `double *d = malloc(sizeof(double)); if (d == NULL) return 1; *d = 3.5; .` Ask: “*what changes between the int and the double version?*” Answer: only the type in the pointer declaration and inside `sizeof` — the shape is identical. This is the “one pattern, many types” payoff.

Board: live failure trace. Ask “*which line would fail if `p` is `NULL`?*” (Line three, the dereference.) If `malloc` returned `NULL`, line three reads from nothing — a crash. The `if` guard makes line two the controlled exit. Correct the habit live: never let a line touch a pointer that *may* be `NULL` before you have checked.

Teaching note: The most common lab mistake this week is dropping the `if (p == NULL)` line “because it never happens.” The teaching move is the trace above: the dereference is not protected, so an out-of-memory condition becomes a crash at a random later line. And do not let students `malloc(sizeof(8))`; the argument is the *type*, and `sizeof` is the type-size operator.

1.7.1 Allocate a complete node

The safer engineering pattern for the whole course:

```
struct node *p = malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 42;
p->next = NULL;
```

Four properties to narrate: `sizeof(struct node)` asks for exactly one node (not a hand-counted 8 or 12); the result is checked before use; `p->data` sets the value field; `p->next = NULL` marks the explicit end. Initialising every field makes state predictable.

Board: audit-the-node checklist (interactive “code review”). Write the four checks and ask the class to *vet a snippet you show*:

1. Was the allocation checked against `NULL`?
2. Was every field initialised before use?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

Show a deliberately broken variant — say, no `NULL` check and no `next = NULL` — and have students find the two violations. Then fix it live.

Board: trace-before-run challenge. Put the two lines `int *p = malloc(sizeof(int)); *p = 80;` on the board and ask: “*what is true after these lines?*” (`p` holds an address; the block holds 80; `p` is not the block.) Reuse Figure 1.5 as the shared reference.

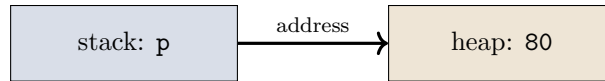


Figure 1.5: What happened: `p` lives on the stack and holds the address of a heap block containing 80.

1.8 How the Heap Grows

Now the converse picture: instead of growing the stack downward, the heap grows *upward*. Three steps, mirroring the stack trace (Figures 1.6–1.8).

Board: heap growth trace (interactive). Draw the empty heap column. Ask a volunteer to “call `malloc(3 * sizeof(int))`” and draw the block holding 10 20 30 just above the bottom; a second volunteer `malloc`’s `q`’s two-element block *above* it; a third volunteers `free(p)` and shades the returned region “free again.” The class watches the used heap grow up then shrink.

Teaching note: The single most instructive contrast in the lecture is running the two traces back to back: a volunteer pushed a stack frame *down* and a volunteer pushed a heap block *up*. Ask “*why opposite directions?*” — so the two regions can grow toward each other and share one free space without colliding (the real layout). If time permits, say only that much; the deck says no more in Week 1.

Board: second distinct instance (prediction). Cover Steps 2–3 and ask: “*after `free(q)`, where does a new `malloc` of three integers land?*” Let guesses land, then reveal that exact placement is the allocator’s choice — what matters is that freed space is *reusable*, not its guessed address. This inoculates students against hard-coding addresses.

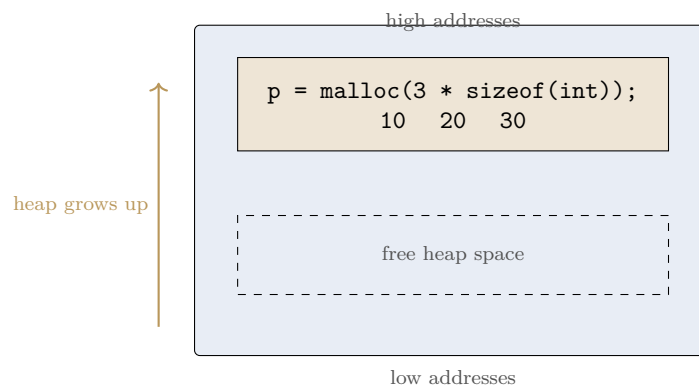


Figure 1.6: Heap growth, Step 1: one `malloc(3 * sizeof(int))` grows the used heap upward with a block holding 10, 20, 30.

1.9 Returning the Memory, and Two Beginner Mistakes

The return idiom is two lines that travel together:

```

free(p);
p = NULL;

```

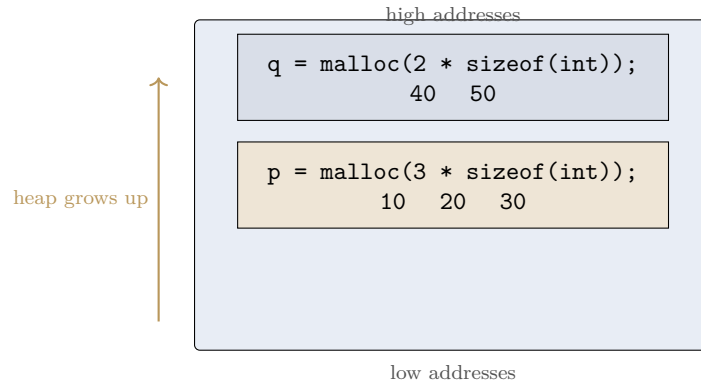


Figure 1.7: Heap growth, Step 2: a second `malloc` places `q`'s block at a higher address; blocks are independent.

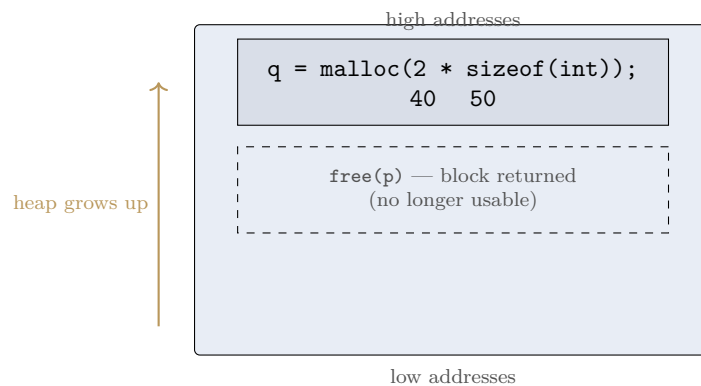


Figure 1.8: Heap growth, Step 3: `free(p)` returns `p`'s block; `q`'s block is untouched.

`free(p)` gives the heap block back to the system; `p = NULL` makes the old pointer visibly empty so it cannot be mistakenly used. This is the top of the deck's summary line: **Request memory, use it, return it.**

The two mistakes that violate it:

- **Memory leak**: forget to call `free`; the block stays reserved (and if the pointer is overwritten, nothing can reach it ever again). Nothing crashes; the program quietly grows.
- **Dangling pointer**: use a pointer after its block was freed. Reads are garbage; writes corrupt whatever now occupies that space.

Board: the two mistakes side by side (interactive clinical case). Draw the heap with one block; then:

1. Sketch the **leak**: erase the variable box while leaving the heap block — ask “*who can still reach this block?*” (No one. It is gone from reach but still reserved.)
2. Sketch the **dangling** pointer: shade the freed block “free,” keep the pointer arrow sharp — ask “*what does the arrow now point at?*” (Nothing usable.)

The live habit: the owner is the one who frees. Ask a student to re-state who owns the node allocated in the `struct node` example.

Board: a second, distinct instance. Show the leak on a *loop*: a program that allocates one node per iteration and never frees any. Trace the growth and ask: “*what eventually happens to the program?*” (It runs out of memory; nothing crashes today, but it dies on a long run.) This makes the leak concrete rather than abstract.

Teaching note: Expect students to free *without* setting NULL, then to dereference the stale pointer in a later function and blame the compiler. The pair `free(p); p = NULL;` is a single habit — teach it as one gesture, not two lines. Also expect the “I freed it but nothing bad happened” shrug: the bug is silent today and fatal on the next run. That silence is why the framework exists.

1.10 Simple Contracts: The ADT

Definition (Abstract data type (ADT)). *A description of what we can do with data, without saying how it is stored: a promise about operations [2].*

The contract states *what*; the implementation states *how*; different implementations can follow the same promise. Sedgewick & Wayne’s one-liner, p.64: a data type is “a set of values and a set of operations on those values” [2].

Board: the bag contract (write it first). Write on the board, one line each:

1. `insert(x)` — put `x` into the bag.
2. `member(x)` — answer whether `x` is present.
3. `size()` — report how many items are present.

Pause and underline: “*the contract does not say whether the bag uses an array or a list.*” That silence is the feature.

Board: one contract, two implementations (interactive classify). Draw the contract box on top, then two boxes below — *dynamic array* and *linked nodes* — with “implements” arrows (Figure 1.9). Run the deck’s classify drill live, having students raise hands:

1. “`member(7)` answers yes or no.” — **contract**: what users can ask.
2. “The values are stored in an array.” — **implementation**: how.
3. “`size()` reports the number of values.” — **contract**: what users can ask.

The split is the lasting skill: **state the contract before writing a line of the implementation.**

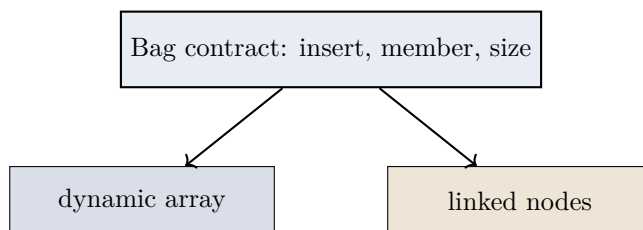


Figure 1.9: One contract, two implementations. Same operations, different internal arrangement; the caller’s code does not change.

Board: second distinct instance (design question). Pose a variant contract — “a queue of integers with `enqueue` and `dequeue`” — and ask the class which of the two bottom boxes they would *guess* is cheaper for *dequeue*. (Either answer is fine; the point is that both *could* implement it, which is the abstract claim.) Note Sedgewick & Wayne’s §1.3, p.120 offers bag/queue/stack as three collections differing only in which object is removed next — a one-minute pointer for the curious.

Board: the course roadmap, ADT by ADT. List on the board: Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Disjoint Sets, Hash Tables, Graphs — the commonly used ADTs, in Karumanchi’s phrasing, “a declaration of data” plus “a declaration of operations” (Karumanchi2017, p. 18–19 PDF) [1]. Point out that this is not a random list: it is exactly the Weeks 3–12 work-list, and every one of those chapters opens the same way this section just did — “What is a Stack?”, then “Stack ADT”, *then* implementation. Today’s bag is the class’s first rehearsal of a pattern the course repeats eight more times.

Teaching note: Karumanchi2017 is a strong anchor for the *shape* of the ADT idea and for this roadmap. It is deliberately not cited in the stack, heap, or pointer sections above. Indexing it found no coverage there of structured programming or of C’s heap-allocation calls (this course’s supporting-books index, Step 0.5 coverage check). When answering student questions: cite Karumanchi2017 for “what is an ADT” and “what’s coming”; cite Wirth2004 for the C memory-management mechanics.

Teaching note: The ADT section is where classes mentally check out unless it is *used*. The classify drill is the anchor — it forces the what-vs-how split out of each student. Do not present the ADT as a vocabulary item; present it as a design decision, and preview that Weeks 3 and 7 implement the *same* stack twice so the choice becomes real. This is deliberately the lightest-weight theoretical point of the lecture, matching the intro-level difficulty.

1.11 Synthesis and Summary

Close by returning to the three prediction boxes from the opening frame and filling them in: *where* = the stack (short-lived locals) and the heap (run-time requests); *how* = `malloc/sizeof` obtains a heap block and `free` returns it, through a pointer handle; *describe* = an ADT states operations before any implementation. The three boxes are the three questions of the title slide, now answered.

Quick reference. *Stack*: automatic, scope-bound, grows down; each call pushes a frame, each return pops it. *Heap*: run-time sized, lifetime-controlled, grows up; `malloc` obtains, `free` returns, NULL means “nothing.” *Pointer*: stores an address; `->` reads a field; never read through NULL. *Allocation idiom*: `malloc + sizeof` → check NULL → initialise every field → use → `free + set NULL`. *Two mistakes*: leak (never freed) and dangling pointer (used after free); every allocation has one owner. *ADT*: operations stated without storage; a data structure is one implementation.

Set up Week 2’s hook: we can now build two implementations of the same bag, but we have no way to say which is better — only “it felt fast.” Next week supplies the vocabulary (asymptotic analysis) to compare them without running either.

References

- [1] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited

in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

- [2] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.
- [3] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.